

Digital Alarm Clock with Radio Project

August - 2023

Table of Contents

Project Expectations.....	3
Circuit Design and Schematic.....	3
Bill of Materials.....	4
Printed Circuit Board.....	5
Enclosure Design.....	8
Testing and Validation.....	13
References.....	15
Appendices.....	16

Project Expectations

The main expected functionalities of the digital clock are being able to set and edit time, display time in both 12-hour and 24-hour format, being able to set and edit alarm that will trigger based on the desired time and the last functionality is to be able to snooze and turn of the alarm through user interface. The main expected functionalities of the Radio are being able to tune to at least one station, play radio output through speaker, be able to control volume through user interface and display the radio information on the display.

Circuit Design and Schematic

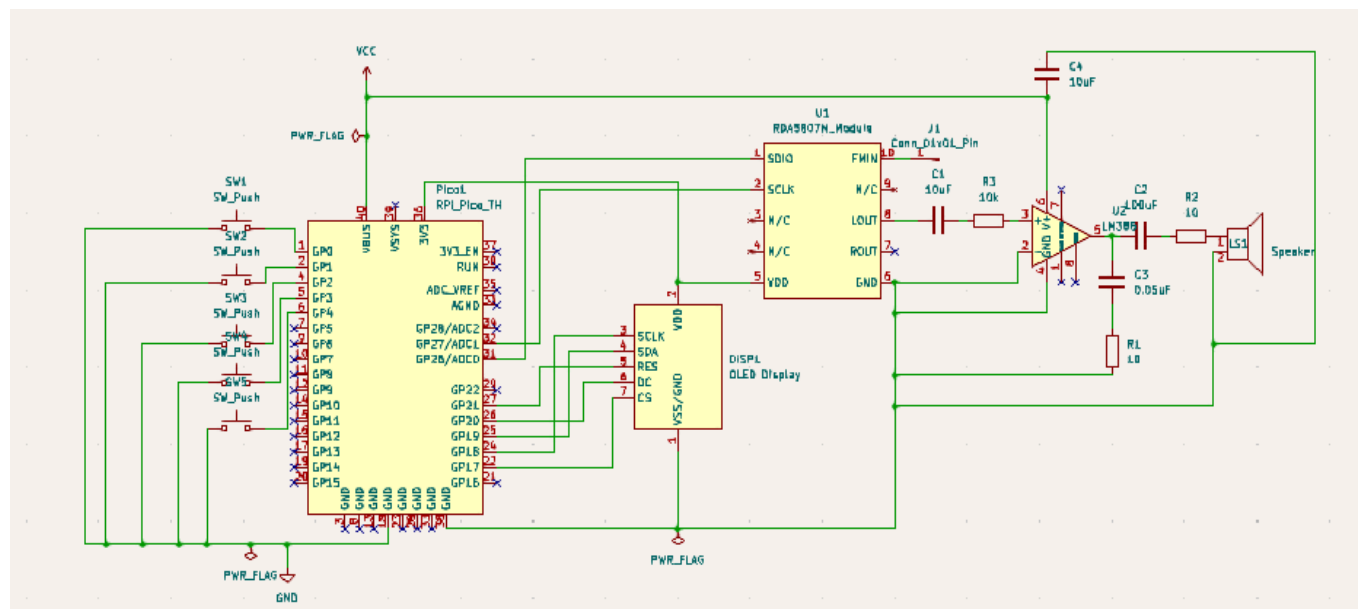


Figure 1. Circuit Schematic Design For Radio and Clock Project.

As shown on Figure 1. Our circuit schematic includes five buttons to control the Pico board, OLED display and a radio receiver. The circuit includes an audio amplifier to amplify the radio waves from the receiver so that it can be heard clearly through the speaker. The five buttons in the circuit each have a certain function. Menu button, Select button, Up button, Down button and Snooze/Mute button.

Bill of Materials

Table 1. Bill of Materials Used to Complete Radio and Clock Project.

Label In Schematic	Part	Quantity	Price (CAD \$)	Total (CAD \$)
RPi_Pico_TH	Raspberry Pico	1	5	5
LM386	LM386N - 4 Audio Amplifier	1	1.5	1.5
RDA5807M_Module	RDA5807M	1	1	1
Speaker	Speaker	1	4	4
OLED Display	1.3" OLED Display SPI	1	4	4
SW_Push	Pushbutton	5	0.25	1.25
C2	100 uF Capacitor	1	0	0
C1,C4	10 uF Capacitor	2	0	0
C3	0.05 uF Capacitor	1	0	0
R3	10k ohm Resistor	1	0	0
R1,R2	10 ohm Resistor	2	0	0
N/A	Micro Usb Cable	1	5	5
N/A	Enclosure	1	15	15
N/A	PCB	1	8	8
				Final Cost: 44.75

We have used all the materials that are shown in Table 1 to complete the whole project which had the final cost of \$44.75 CAD.

Printed Circuit Board

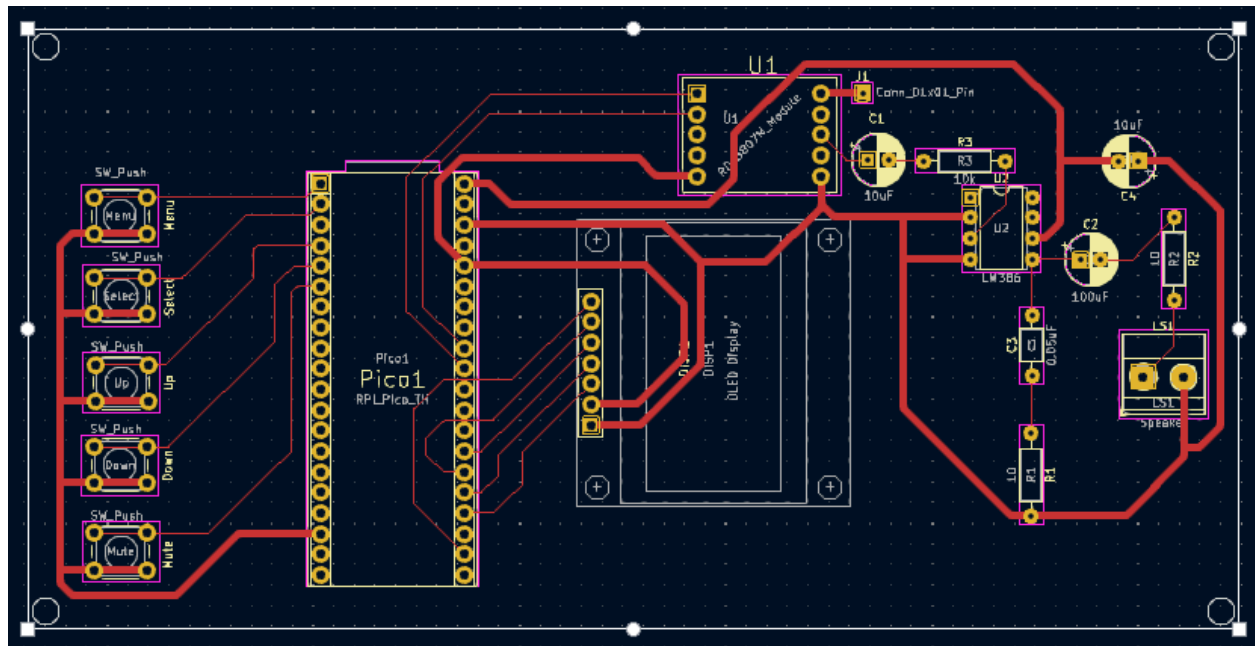


Figure 2. Printed Circuit Board Design For Radio and Clock Project.

As shown on Figure 2. The traces of the circuit are all on the top layer to make it simple and easier to solder everything. The power and ground pins are all connected with thicker traces than the rest of the circuit to make sure the whole circuit is powered.

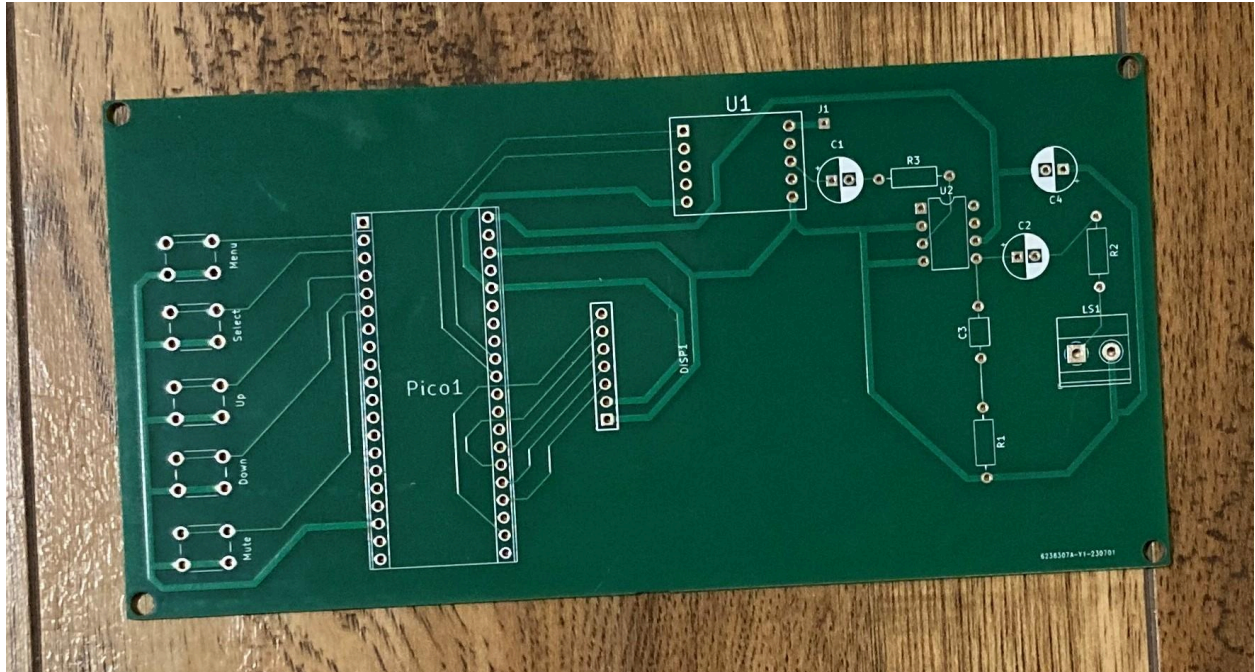


Figure 3. Front Side of Printed Circuit Board Before Soldering Components.

Figure 3 is the front side of the printed pcb board, note the thicker traces for GND, 3.3V and 5V allowing for higher current and reliability.

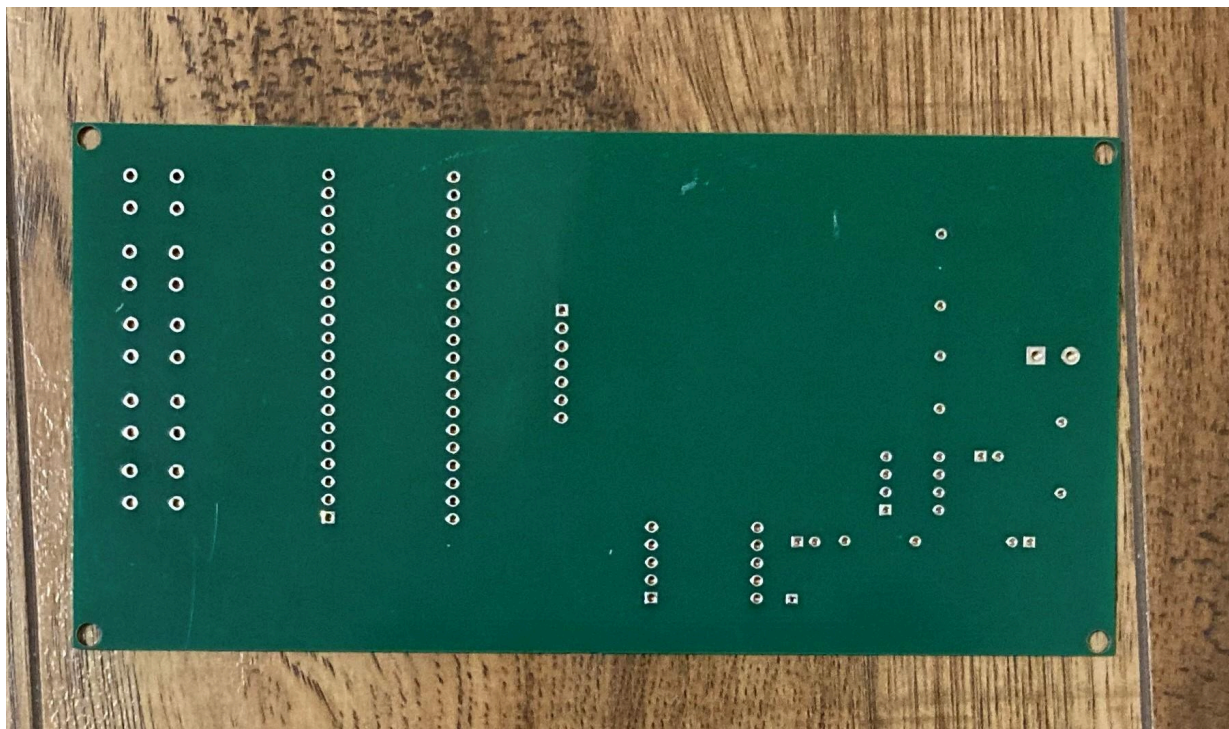


Figure 4. Back Side of Printed Circuit Board Before Soldering Components.

Figure 4 is the back side of the printed pcb board, as noted above there are no traces on this side.

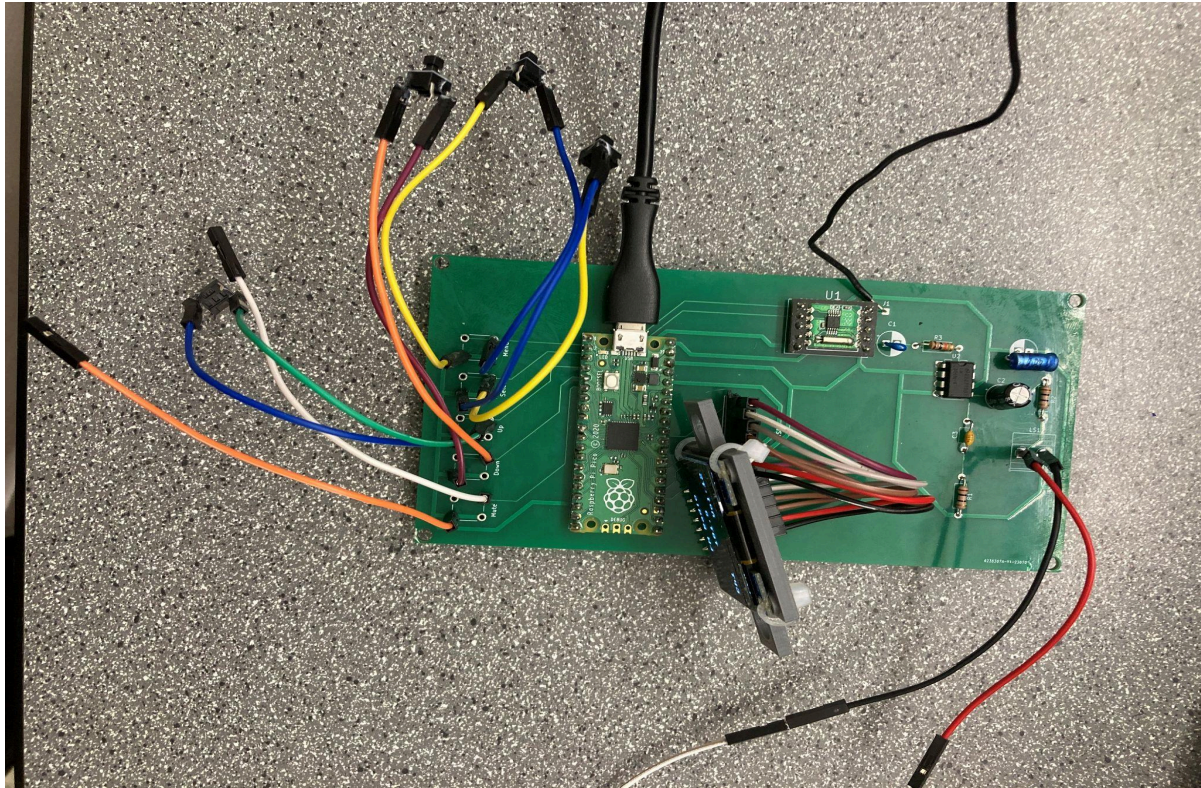


Figure 5. Front Side of Printed Circuit Board After Soldering Components.

Figure 5 is the front side of the PCB after being populated with all components. The wires allow the connected components to be mounted away from the pcb on the enclosure.

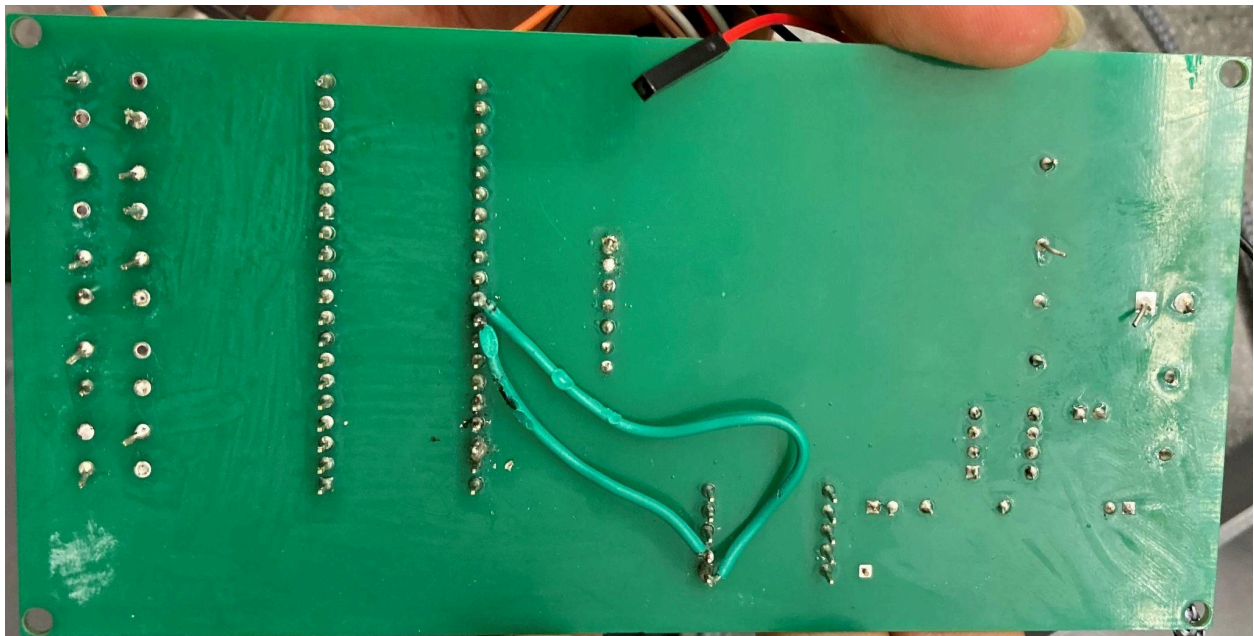


Figure 6. Back Side of Printed Circuit Board After Soldering Components.

As shown on Figure 6. The traces of SDA and SCK pins on the audio receiver were too thin, so the connection wasn't working all the time and the software would crash sometimes, so we added two wires for the SDA and SCK pins at the bottom of the PCB, so that the connection is better and the radio receiver works as expected.

Enclosure Design

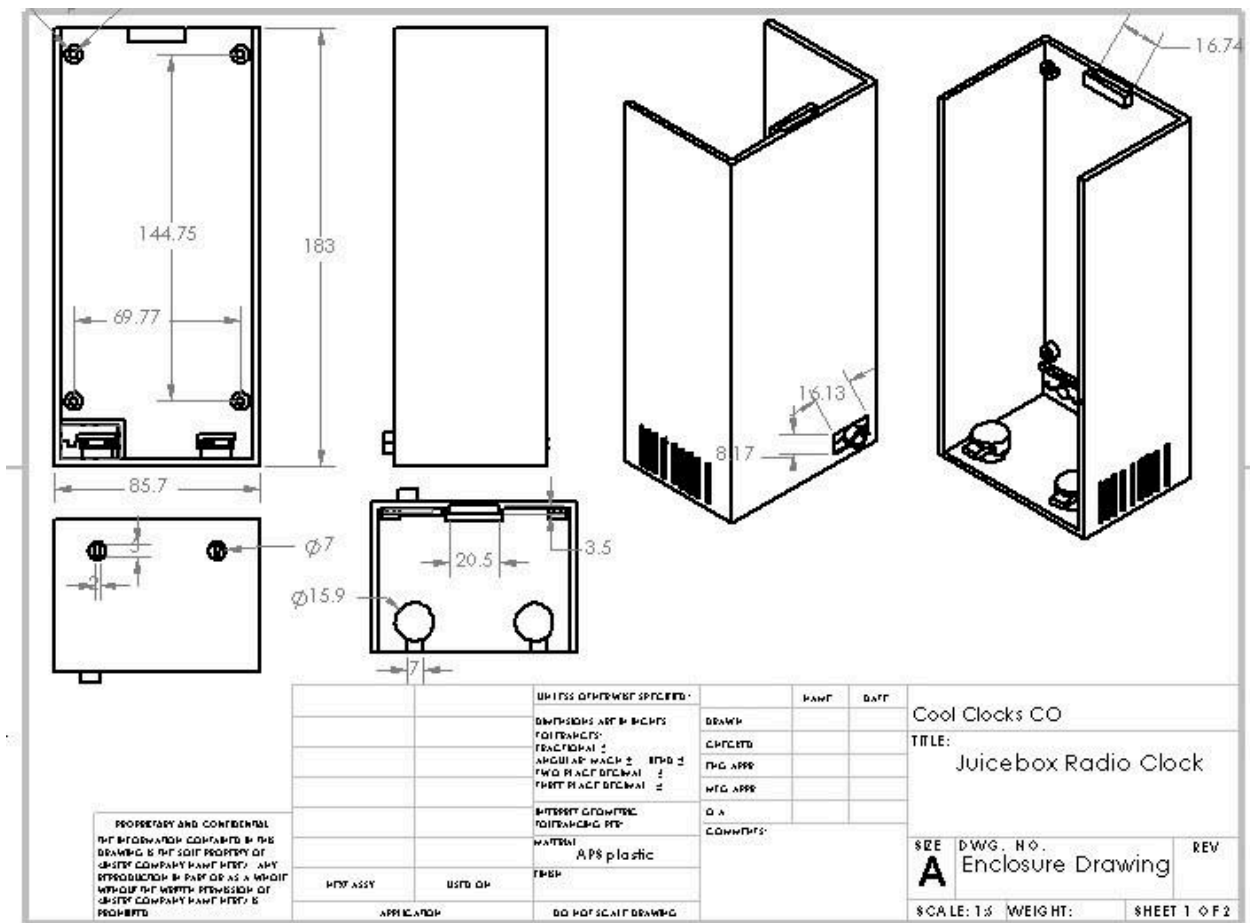


Figure 7. Back portion of the main enclosure drawing.

Figure 7 is a drawing of the back portion of the enclosure. Many of the parts in Figure 8 are individual parts but have been assembled in the drawing. The circular standoffs from the inside back of the box holds the breadboard to the back. The tunnel coming out the back of the enclosure seals the enclosure after letting the power cable through. The two round caps on the bottom of the enclosure contain a pin with an extruding rectangular piece. The pin can be rotated about 180 degrees. The rectangular piece acts as a latch that rotates into c shaped brackets on the front enclosure to secure the front and back together. The qr code-like pattern (like one would find on a juice box) on the side of the bottom enclosure consists of many rectangle holes next to

each other. The speaker goes behind the qr code pattern; this design sacrificed some audio quality as the speaker did not face towards the user, but improved the aesthetic of the design.

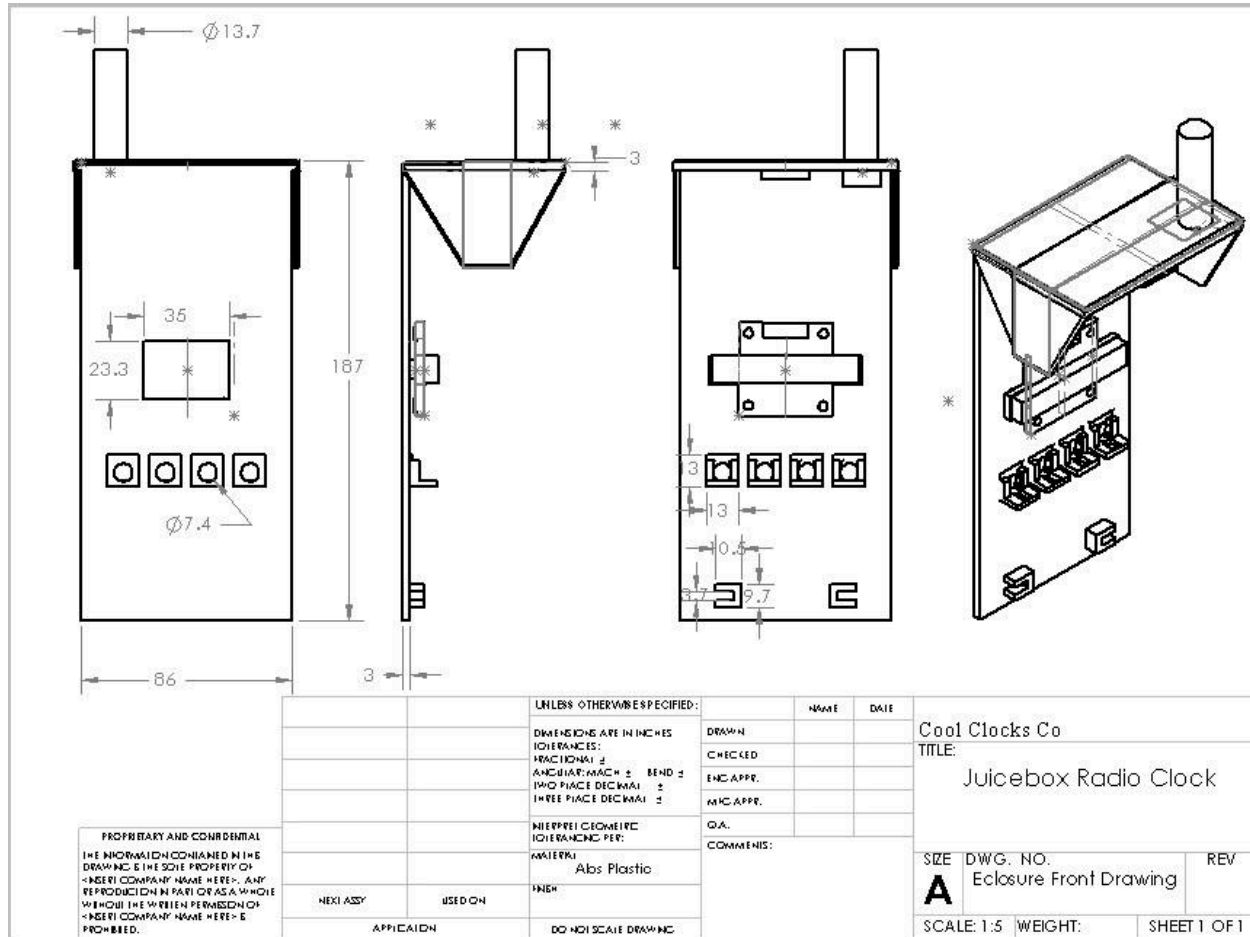


Figure 8. Front portion of main enclosure drawing.

Figure 8 is the drawing of the front portion of the main enclosure; it is the assembly of many parts. The largest hole on the front is the space for the oled and the four small holes allow for the buttons to come out. The rod or “straw” on top of the enclosure is put on a button. The straw was made into a button as it seemed like a fun way to implement the juicebox design and made the snooze/mute button stand out from the rest.. The holes are rounded circles as the buttons had caps printed for them. The two c brackets on the lower end of the front enclosure have latches that turn into them from the bottom part of the enclosure. The c brackets and latches secure the two parts together.



Figure 9. Back of front enclosure with all components mounted.

Figure 9 is a picture of the inside of the front of the enclosure with all components attached. The oled and the buttons were attached to the pcb by cables. A struggle was ensuring the case had enough space inside to accommodate the cables. The space issue was resolved by ensuring the pcb was zip tied to the back standoffs and using latches to hold the enclosures together.

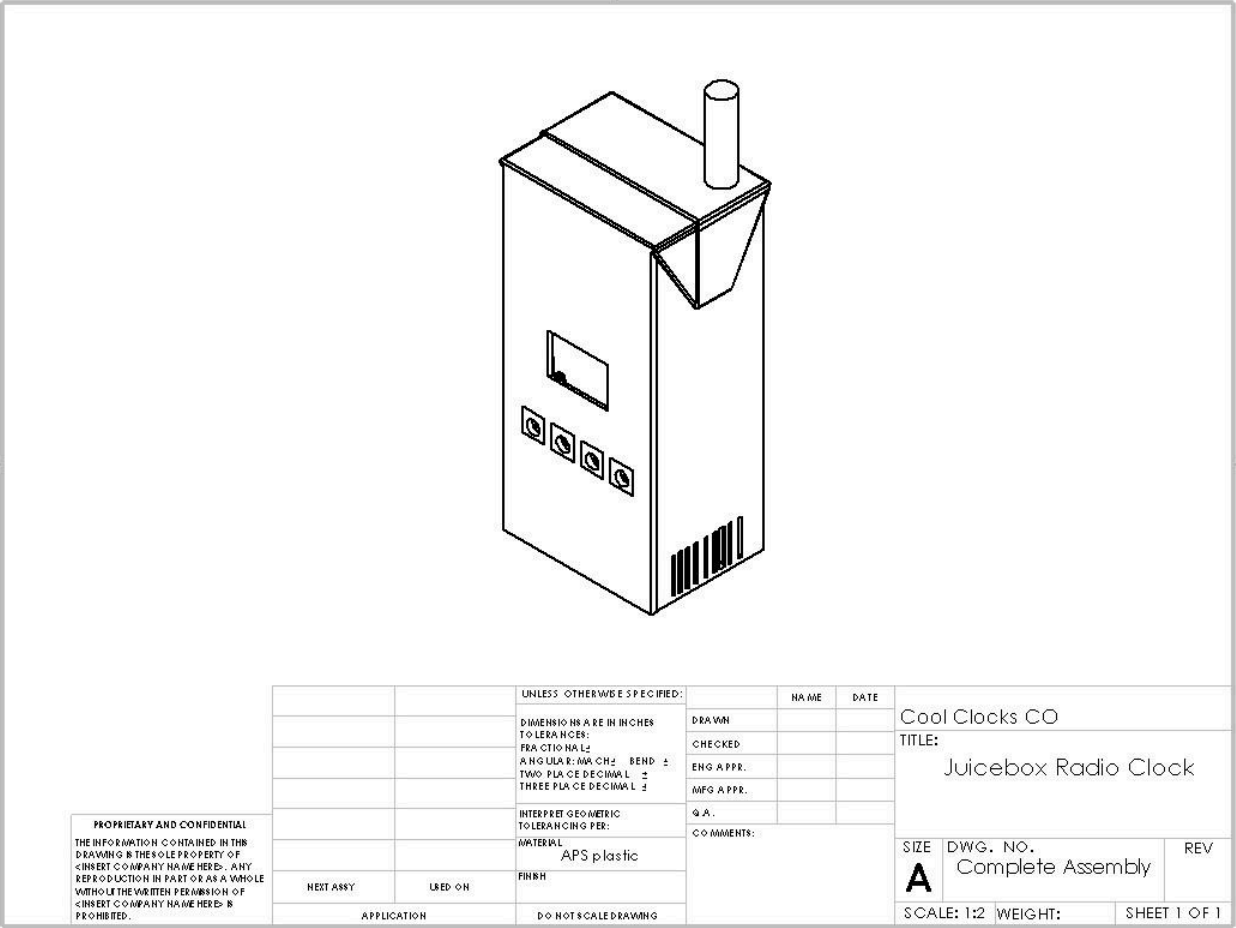


Figure 10. Entire assembly of the enclosure drawing.

Figure 10 is a drawing of the entire enclosure assembled. This is the front and back of the enclosure combined where the back part of the enclosure slides into the front.



Figure 11. Front of final assembly with all components attached.

Figure 11 is a picture of the printed assembly with all parts integrated. Note the design had to be changed from the drawings in practice. The bracket used to hold the straw button to the top of the enclosure put too much stress on the button, breaking it. The solution was to just glue the straw onto the top of the enclosure and move the snooze/mute functionality to the other buttons using software.

Testing and Validation

Table 2. List of Tests Conducted to Validate Project

Tests	Procedure	Expectations
Display	• Connect Micro USB cable to Pico	Display turns on with time starting at 23:59
Set Time	• Push Menu Button • Select Set Time option using Up and Down buttons to navigate options • Set Time and push the select button to finalize time	Display shows time selected
Switching Between 24/12 Hour Format	• Push Menu Button • Select 24/12 hr option using Up and Down buttons to navigate options	Display shows time in 24 hour or 12 hour format depending on what it was before
Set and Trigger Alarm	• Push Menu Button • Select Set Alarm option using Up and Down buttons to navigate options • Set Alarm and push the select button to finalize the time	Display shows “ALARM ON” text and audio comes out

Snooze and Turn OFF Alarm	● Press Snooze button to stop the alarm	Alarm turns off and Display goes back to showing time
Tuning Radio and Playing Audio Through Speaker	● Push Menu Button ● Select Radio Mode option using Up and Down buttons to navigate options.	Radio tunes to 101.9 FM station and audio is heard from speaker
Volume Control	● Push Up button while radio is playing ● Push Down button while radio is playing	Volume of radio will go up and down depending on the buttons pushed
Display Radio Channel Information	● Push Menu Button ● Select Radio Mode option using Up and Down buttons to navigate options.	Display shows the radio station 101.9 FM with volume on the bottom right of the screen

References

- [1] T.C. Ilamparithi. (2023). Timer Interrupt Button Circuit [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2263671/View> (accessed July 20, 2023)
- [2] T. C. Ilamparithi. (2023). Timer Interrupt Button Circuit [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2271292/View> (accessed July 20, 2023)
- [3] T. C. Ilamparithi, Lab 2: Increment number with button push and display it on OLED screen [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2262784/View> (accessed July 20, 2023).
- [4] University of Victoria. Lab 3 Schematic [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2262792/View> (accessed July 26, 2023)
- [5] T. C. Ilamparithi, Lab 3: Volume control of the radio module [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2262797/View> (accessed July 26, 2023).
- [6] Texas Instruments, LM386 Low Voltage Audio Power Amplifier [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm386.pdf> (accessed July 26, 2023)
- [7] T. C. Ilamparithi, Audio Amplifier Circuit [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2252559/View> (accessed July 26, 2023)
- [8] T. C. Ilamparithi, Lab 4: Introduction to KiCAD [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2282692/View> (accessed July 26, 2023).
- [9] T. C. Ilamparithi, Getting Started With SolidWorks [Online]. Available: <https://bright.uvic.ca/d2l/le/content/278802/viewContent/2298404/View> (accessed July 28, 2023).
- [10] Engineers and Geoscientists British Columbia. “Code Of Ethics.” egbc.ca [Online]. Available: <https://www.egbc.ca/Complaints-Discipline/Code-of-Ethics/Code-of-Ethics> (accessed August 5, 2023)

Appendices

```
16
17 #Buttons
18 Menu_Button = machine.Pin(0,machine.Pin.IN,machine.Pin.PULL_UP) #C
19 Set_Button = machine.Pin(1,machine.Pin.IN,machine.Pin.PULL_UP)
20 Up_Button = machine.Pin(2,machine.Pin.IN,machine.Pin.PULL_UP)
21 Down_Button = machine.Pin(3,machine.Pin.IN,machine.Pin.PULL_UP)
22 Snooze_Button = machine.Pin(4,machine.Pin.IN,machine.Pin.PULL_UP)
23
```

Figure 12. Code to sync variables with GPIO pins

```
29 def Check_Button(Button): #checks to see if a button was pressed (if button is held down it only records one instance of the button being pressed)
30
31     if(Button.value() == 0 and flag == 0): #checks to see if button is pressed and if it was released
32         time.sleep(0.065) #pauses code 0.065 seconds
33         if(Button.value() == 0): #checks again to see if button is pressed #if so
34             flag = 1 #sets flag to be 1
35             while(Button.value() == 0): #waits until button released
36                 #nothing
37             flag = 0 #flag set to zero
38             time.sleep(0.05) #wait 0.05 seconds
39             return True #returns true (button was pressed)
40     return False #returns false (button not pressed)
41
42
43 def Check_Button_Hold_EN(Button): #checks to see if button pressed (if button is held down it records infinitely many button presses)
44
45     if(Button.value() == 0): #if the button is pressed
46         time.sleep(0.065) #wait 0.065 seconds
47         if(Button.value() == 0): #if the button is still pressed
48             return True #return true (button was pressed)
49     return False #button was not pressed
50
51
```

Figure 13. Code to check buttons to see if pressed and debounce them

```

def Clock_Menu(): #clock menu
    global Device_State #grabs the Device_State global variable
    Select = 0

    while(Select != 999): #FSM tha updates the screen based on the buttons pressed
        oled.fill(0)

        if(Check_Button(Up_Button)):
            Select += 1
            if(Select > 2):
                Select = 0

        if(Check_Button(Down_Button)):
            Select -= 1
            if(Select < 0):
                Select = 2

        if(select == 0):
            if( Check_Button(Set_Button)):
                Set_Time()
                Select = 999

        elif(select == 1):
            if( Check_Button(Set_Button)):
                Set_Alarm()
                Select = 999

        elif(select == 2):
            if( Check_Button(Set_Button)):
                Device_State = "Radio"
                Select = 999

```

Figure 14. Rudimentary menu select code allowing the user to pick between states